UNIVERSIDAD DE GRANADA

Universidad de Granada
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Grado en Ingeniería Informática

Dirección y Gestión de Proyectos (DGP)
Curso Académico 2026–2027

MANUAL DE ACCESIBILIDAD Y CALIDAD

Proyecto AUTOVIDA

Formación y prácticas para el equipo de desarrollo

Sistema de agenda y asignación de tareas accesible para alumnado NEAE / PTVAL

Equipo:
CLENCH Software Development

Responsables de este manual:
Julián Carrión Tovar Accesibilidad
Yeray Rodríguez Navas Calidad

Equipo del proyecto:
Miguel Ángel Luque Gómez
Pablo Hernández Ibáñez
Pablo de la Torre Roldán
José Rodríguez Fernández
Amine Azzammouri

Índice

1. Propósito y alcance del manual **1**
1.1. Objetivos de aprendizaje . . . 1
1.2. Principio de trabajo . . . 1

2. Accesibilidad en el frontend: diseño visual **1**
2.1. Colores y contraste . . . 2
2.2. Tipografía legible . . . 2
2.3. Revisión visual rápida . . . 2
2.4. Referencia de accesibilidad . . . 2

3. Accesibilidad en el frontend: interacción y contenido **3**
3.1. Evitar ventanas emergentes . . . 3
3.2. Texto y mensajes comprensibles . . . 3
3.3. Teclado y foco . . . 3
3.4. Ejercicio para el equipo . . . 3

4. Formación básica de backend **3**
4.1. Qué es el backend . . . 4
4.2. Conceptos esenciales . . . 4
4.3. Flujo que debe entender todo el equipo . . . 4

5. Buenas prácticas de backend **4**
5.1. Datos, validación y consistencia . . . 4
5.2. Permisos, seguridad y privacidad . . . 5
5.3. Errores y disponibilidad . . . 5
5.4. Práctica guiada . . . 5

6. Prácticas de calidad para el equipo **5**
6.1. Ciclo de trabajo recomendado . . . 5
6.2. Reglas de código . . . 6
6.3. Reglas de documentación . . . 6
6.4. Niveles de prueba . . . 6
6.5. Criterio de terminado (*Definition of Done*) . . . 7
6.6. Gestión de incidencias . . . 7

7. Plan de formación y revisión **7**
7.1. Bloque 1: interfaz accesible . . . 7
7.2. Bloque 2: recorrido de backend . . . 7
7.3. Bloque 3: calidad compartida . . . 7
7.4. Ritmo de revisión . . . 8

8. Responsabilidades y colaboración **8**
8.1. Cómo colaborar ante un desacuerdo . . . 8

9. Lista de comprobación y mejora continua **8**
9.1. Interfaz y accesibilidad 8
9.2. Backend y calidad 9
9.3. Registro de formación y decisiones 9
9.4. Referencias para ampliar la formación 9

1. Propósito y alcance del manual

Este manual reúne prácticas y una formación inicial para que el equipo de AUTOVIDA incorpore accesibilidad y calidad durante el diseño, la implementación y la revisión del sistema. Está dirigido a todas las personas del equipo, no solo a quienes programan: las decisiones de contenido, diseño, pruebas y documentación también afectan a la experiencia de uso.

La primera versión desarrolla las pautas acordadas hasta ahora para la interfaz (colores, ausencia de ventanas emergentes, legibilidad del texto y compatibilidad con opciones de acceso como el barrido), además de criterios de accesibilidad para el backend y controles de calidad. Se ampliará conforme se definan nuevas necesidades con el centro y con las personas usuarias.

1.1. Objetivos de aprendizaje

Al terminar la formación, cada integrante debería poder:

- reconocer decisiones visuales y de interacción que pueden dificultar el uso;
- aplicar una paleta con contraste suficiente y no depender solo del color para comunicar;
- evitar interrupciones inesperadas, mantener los textos breves y comprobar que el barrido permite seleccionar correctamente los botones;
- permitir ajustar el tamaño del texto y de los botones, tanto para ampliarlos como para reducirlos, sin perder legibilidad ni facilidad de selección;
- proporcionar a cada botón un nombre accesible claro y añadir texto alternativo a los iconos cuando corresponda, para que las tecnologías de apoyo anuncien su función;
- aplicar criterios de accesibilidad también en el backend;
- revisar los cambios con criterios acordados, registrar incidencias y comprobar que se han resuelto.

1.2. Principio de trabajo

La accesibilidad y la calidad se incorporan desde el principio, no se reservan para una revisión final. Las pautas de este documento son una base de trabajo; no sustituyen las pruebas con alumnado y profesionales del centro ni la validación de requisitos concretos.

Cómo utilizar este manual

Antes de implementar una funcionalidad, el equipo revisará las pautas aplicables y acordará cómo comprobarlas. En la revisión del cambio se anotarán las evidencias, los problemas detectados y quién se encarga de resolverlos.

2. Accesibilidad en el frontend: diseño visual

El frontend es la parte de AUTOVIDA con la que interactúan el alumnado y el personal del centro. Debe presentar información comprensible y predecible, y permitir que las tareas principales se identifiquen sin depender de la memoria, de una percepción concreta del color o de instrucciones complejas.

2.1. Colores y contraste

- **Definir una paleta reducida y consistente.** Asignar colores con funciones claras (por ejemplo, acción principal, información, aviso y error) y reutilizar esas funciones en todas las pantallas.
- **Comprobar el contraste entre texto y fondo.** Como referencia de diseño, buscar al menos una relación de contraste de 4,5:1 para texto normal y 3:1 para texto grande. No decidir a simple vista: utilizar un comprobador de contraste sobre los colores reales y revisar también estados como deshabilitado, seleccionado y al pasar el foco.
- **No comunicar solo mediante el color.** Acompañar los estados de una etiqueta, un icono reconocible o una explicación textual. Por ejemplo, una tarea urgente no debe distinguirse únicamente por aparecer en rojo.
- **Acompañar los botones con símbolos reconocibles.** Combinar el símbolo con una etiqueta de texto clara y mantener el mismo símbolo para la misma acción; así, la función del botón se entiende aunque no se perciba el color. El símbolo no debe sustituir la etiqueta textual.
- **Evitar combinaciones difíciles de diferenciar.** Revisar especialmente texto y controles en rojo/verde, tonos muy próximos y fondos con imágenes o degradados.
- **Mantener la coherencia.** El mismo color debe significar lo mismo en distintas vistas; los colores decorativos no deben competir con la información ni reducir su contraste.

2.2. Tipografía legible

- Elegir una tipografía sencilla y familiar, con formas de letras claramente diferenciables.
- Usar un tamaño cómodo en el dispositivo objetivo y permitir que el texto se amplíe o se reduzca sin que se corte ni se solape. Permitir también ajustar el tamaño de los botones, conservando etiquetas legibles y un área fácil de seleccionar. Revisar la interfaz en móvil y escritorio.
- Separar títulos, instrucciones y acciones mediante jerarquía visual, espacio en blanco y alineación consistente; evitar bloques densos y líneas excesivamente largas.
- Preferir frases directas, vocabulario cotidiano y una instrucción por frase. Acompañar palabras poco conocidas con una explicación breve.
- Reservar las **MAYÚSCULAS** para rótulos breves, siglas o avisos puntuales. No redactar instrucciones ni párrafos completos en mayúsculas: las frases en caja mixta suelen ser más fáciles de leer.

2.3. Revisión visual rápida

Antes de aceptar una pantalla, comprobar: ¿se lee cada texto sobre su fondo?, ¿se entiende el estado sin mirar el color?, ¿se distinguen las acciones por sus símbolos y etiquetas?, ¿se mantienen legibles los textos y fáciles de seleccionar los botones al cambiar su tamaño?, ¿se puede entender cada instrucción en una lectura?

2.4. Referencia de accesibilidad

El equipo tomará las Pautas de Accesibilidad para el Contenido Web (WCAG) 2.2, nivel AA, como referencia técnica para los criterios que correspondan a la interfaz. La conformidad no se dará por supuesta: se comprobarán los criterios aplicables y se harán pruebas con personas usuarias. La referencia principal para contraste es el criterio 1.4.3 de WCAG.

3. Accesibilidad en el frontend: interacción y contenido

3.1. Evitar ventanas emergentes

Las ventanas emergentes inesperadas interrumpen la tarea, pueden ocultar el contenido y dificultan que algunas personas comprendan qué ha ocurrido. En AUTOVIDA se evitarán las ventanas emergentes (*pop-ups*) automáticas, publicitarias o innecesarias. La información habitual debe aparecer en el flujo de la propia pantalla, cerca de la acción a la que se refiere.

Para avisos no urgentes, utilizar mensajes integrados que no roben el foco ni desaparezcan antes de que puedan leerse. Para confirmar que una tarea se ha guardado, mostrar una confirmación breve en contexto. Para errores, conservar los datos introducidos y explicar cómo corregirlos junto al campo correspondiente.

Si una decisión verdaderamente importante requiere bloquear temporalmente la tarea, el equipo debe justificarlo y diseñar un diálogo accesible: explicar claramente el motivo y las opciones, dar nombres comprensibles a los botones, permitir cerrar o cancelar cuando sea seguro, mantener el foco dentro del diálogo mientras esté abierto y devolverlo al elemento que lo abrió al cerrarlo. No se abrirán diálogos por el mero hecho de cargar una página ni para mostrar información rutinaria.

3.2. Texto y mensajes comprensibles

- Indicar qué se espera de la persona con verbos concretos: por ejemplo, “Guardar tarea” en vez de “Aceptar” cuando esa sea la acción real.
- Usar los mismos términos para la misma función en toda la aplicación; no alternar sin motivo entre sinónimos.
- Explicar errores en lenguaje claro, decir qué elemento necesita atención y proponer el siguiente paso. Evitar mensajes como “Error 400” sin explicación útil.
- No depender de iconos sin etiqueta ni usar únicamente mayúsculas, color o sonido para transmitir una instrucción.
- Dar instrucciones antes de una acción cuando sean necesarias y confirmar después el resultado de forma visible y persistente el tiempo suficiente para leerlo.

3.3. Teclado y foco

Aunque la aplicación se dirija principalmente a alumnado que utiliza pantalla táctil, los controles deben tener un orden de navegación predecible y un indicador visible de foco cuando se usan con teclado. Los botones y campos deben ser controles reales y etiquetados, no elementos visuales que solo funcionen con ratón. Al abrir o cerrar un diálogo excepcional se comprobará el comportamiento del foco descrito anteriormente.

3.4. Ejercicio para el equipo

En parejas, revisar una pantalla o prototipo y localizar: un mensaje que podría ser más claro, un estado que dependa solo del color y una posible interrupción. Proponer un cambio y explicar cómo probarlo con una persona usuaria o mediante una lista de comprobación.

4. Formación básica de backend

4.1. Qué es el backend

El backend es la parte que procesa la información y las reglas de AUTOVIDA fuera de la pantalla que ve la persona usuaria. De forma simplificada, la interfaz envía una petición; el servidor comprueba que sea válida y que la persona tenga permiso; consulta o actualiza los datos almacenados; y devuelve un resultado que la interfaz presenta de forma comprensible.

Por ejemplo, al guardar una tarea, el servidor debe comprobar que los campos requeridos tienen valores válidos, que quien la guarda puede modificarla, y que la operación se ha almacenado correctamente. La interfaz puede ayudar a prevenir errores, pero nunca debe ser la única barrera: los datos recibidos por el servidor se consideran no confiables hasta validarlos.

4.2. Conceptos esenciales

- **API:** conjunto definido de operaciones por las que la interfaz y el servidor intercambian datos. Cada operación debe tener entradas y respuestas documentadas.
- **Base de datos:** almacenamiento organizado de la información. Debe evitar registros inconsistentes y guardar solo los datos necesarios para las funciones acordadas.
- **Autenticación y autorización:** la autenticación identifica quién accede; la autorización determina qué acciones puede realizar esa persona. Son comprobaciones distintas.
- **Validación:** reglas aplicadas a los datos recibidos, como campos obligatorios, formatos y límites permitidos. Deben comprobarse en el servidor.
- **Respuesta y error:** resultado que el servidor comunica a la interfaz. Debe ser consistente y permitir mostrar una explicación útil sin revelar información interna.

4.3. Flujo que debe entender todo el equipo

Para cada función, dibujar o describir estos pasos: acción del usuario, petición enviada, validaciones, regla de negocio, lectura o escritura de datos, respuesta y mensaje visible. Este esquema ayuda a detectar requisitos olvidados y a planificar pruebas aunque no todas las personas programen backend.

Idea clave

El frontend facilita una interacción comprensible; el backend protege las reglas y los datos. Nunca se debe confiar en que la interfaz haya enviado información correcta o de que una persona tenga permiso solo porque un botón estaba oculto.

5. Buenas prácticas de backend

Estas pautas son una introducción independiente del lenguaje o la arquitectura que finalmente utilice AUTOVIDA. Se concretarán cuando el equipo acuerde la tecnología, los perfiles y los datos que realmente necesita el proyecto.

5.1. Datos, validación y consistencia

- Validar cada entrada en el servidor: presencia de campos requeridos, tipo, longitud y valores permitidos. Rechazar lo inválido con un mensaje controlado y comprensible para la interfaz.
- Aplicar las reglas de negocio en un lugar consistente. No duplicar reglas contradictorias entre pantallas y servidor.

- Guardar únicamente los datos necesarios para las funciones aprobadas. Usar nombres y formatos consistentes y establecer relaciones claras entre usuarios, tareas y actividades.
- Evitar estados imposibles o duplicados mediante restricciones y operaciones de escritura seguras. Si una operación debe completarse como un todo, impedir que quede guardada a medias.
- Definir cómo se corregirán errores de datos y cómo se harán copias de seguridad y restauraciones, según el alcance y la infraestructura aprobados.

5.2. Permisos, seguridad y privacidad

- Comprobar permisos en cada operación protegida, no solo en la interfaz. Aplicar el mínimo acceso necesario para cada perfil.
- No incluir contraseñas, tokens, claves ni datos personales innecesarios en el código, mensajes de error o registros técnicos. Las credenciales se gestionan mediante la configuración segura del entorno elegido.
- No devolver información de otras personas al consultar o modificar una tarea. Probar expresamente los casos de acceso permitido y denegado.
- Tratar los datos del alumnado con especial cuidado: limitar su uso al propósito acordado, restringir quién puede verlos y no reutilizarlos en pruebas sin autorización y protección adecuada.
- No mostrar trazas internas, consultas, rutas del servidor o detalles técnicos a las personas usuarias. Registrar solo lo necesario para investigar fallos y con acceso limitado.

5.3. Errores y disponibilidad

El servidor debe distinguir entre una operación completada, una entrada incorrecta, falta de permisos y un fallo inesperado. La interfaz recibirá una respuesta previsible y podrá explicarle a la persona qué ha pasado y qué puede hacer. Los fallos técnicos no deben borrar silenciosamente datos ni presentarse como si la tarea se hubiera guardado.

El equipo identificará las operaciones importantes, probará sus casos normales y de error y documentará las dependencias de las que dependen. No se prometerá disponibilidad, recuperación o seguridad absoluta: se acordarán medidas concretas, se probarán y se anotarán sus límites.

5.4. Práctica guiada

En grupos, recorrer el flujo “crear una tarea”. Enumerar datos de entrada, reglas de validación, quién puede crearla, qué se guarda, respuestas posibles y qué mensaje muestra la interfaz si no se puede guardar. Revisar que no se confunda un fallo con una operación completada.

6. Prácticas de calidad para el equipo

La calidad del software asegura que el producto responde a los requisitos acordados y que el sistema resultante es comprensible, revisable y mantenible. No significa que una sola persona inspeccione todo al final del proceso: cada integrante verifica su propio trabajo y el equipo comparte la responsabilidad del resultado final.

6.1. Ciclo de trabajo recomendado

1. **Acordar el resultado.** Describir la necesidad con lenguaje claro y definir los criterios de aceptación para reconocer que está resuelta.

2. **Diseñar la comprobación.** Identificar y anotar los casos de uso normales, los flujos de error previsibles y los requisitos de accesibilidad aplicables antes de comenzar la implementación.
3. **Implementar cambios atómicos.** Mantener cada aportación (*commit*) centrada en una única funcionalidad o corrección para facilitar su revisión y reducir conflictos.
4. **Revisar y probar.** Otra persona del equipo comprueba el cambio mediante *Pull Requests* o revisiones por pares, ejecuta las pruebas disponibles y verifica la lista de accesibilidad y calidad.
5. **Registrar el resultado.** Anotar las pruebas realizadas, incidencias detectadas, decisiones técnicas y tareas pendientes. No se debe marcar como terminado ningún elemento que mantenga un bloqueo conocido o carezca de revisión.

6.2. Reglas de código

- **Convenciones de nomenclatura:** Utilizar nombres descriptivos, explícitos y en un idioma consensuado para variables, funciones y clases. Evitar abreviaturas ambiguas o nombres genéricos. Se utilizará la nomenclatura *camelCase* para el código y *snake_case* para los archivos subidos al Github de CLENCH.
- **Formato y análisis estático:** Emplear herramientas de *linting* y formateo automático para garantizar que todo el código del proyecto mantenga un estilo uniforme, independientemente de quién lo escriba.
- **Modularidad y simplicidad:** Aplicar el principio de responsabilidad única (funciones que realicen una sola tarea) y evitar la duplicación de código.
- **Gestión de dependencias:** Evitar la inclusión de librerías externas para resolver problemas triviales que puedan implementarse de forma nativa y segura.

6.3. Reglas de documentación

- **Comentarios en el código:** Se utilizarán nombres claros y descriptivos para variables, funciones y clases antes de recurrir a comentarios. Los comentarios deben reservarse para explicar el "porqué" de las decisiones complejas, lógicas de negocio específicas o parches inusuales.
- **Sincronización técnica:** El código y su documentación deben actualizarse al mismo tiempo. No se permitirá una documentación desactualizada.
- **Claridad y formato:** Redactar con un lenguaje directo y estructurado. Toda documentación externa o manual de usuario final debe validarse bajo criterios de lectura fácil y accesibilidad cognitiva.

6.4. Niveles de prueba

- **Prueba unitaria:** Comprueba una regla pequeña y específica de manera aislada (por ejemplo, validar que un campo obligatorio del modelo no acepte valores nulos).
- **Prueba de integración:** Verifica que los componentes conectados colaboren correctamente (por ejemplo, que el envío de un formulario actualice la base de datos y devuelva el código de estado esperado).
- **Prueba de flujo completo (End-to-End):** Recorre una tarea desde la interacción en la interfaz gráfica hasta la persistencia en el servidor, evaluando el sistema tal como lo experimentaría la persona usuaria.

- **Prueba de aceptación:** Contrasta el producto final con la necesidad inicial y los criterios acordados internamente y, cuando proceda, validándolo directamente con el centro o los usuarios destinatarios.

Las comprobaciones automáticas son fundamentales para prevenir regresiones, pero no sustituyen la revisión visual humana, la comprobación de navegación por teclado, el uso de lectores de pantalla ni las pruebas empíricas con las personas destinatarias.

6.5. Criterio de terminado (*Definition of Done*)

Una tarea se considera terminada exclusivamente cuando: cumple los criterios de aceptación acordados, cuenta con las pruebas pertinentes superadas, el código ha sido revisado y aprobado por otro miembro del equipo, no existen defectos bloqueantes ocultos y toda la documentación técnica y de usuario afectada ha sido actualizada. Si alguna condición excepcional no aplica, debe documentarse explícitamente el motivo.

6.6. Gestión de incidencias

Toda incidencia (o *bug*) reportada debe documentar: los pasos exactos para reproducirla, el resultado esperado, el resultado observado, la gravedad del impacto, evidencias (capturas o *logs* sin exponer datos personales o sensibles), la persona asignada y el estado actual.

El orden de prioridad en la resolución debe ser: primero los bloqueos de funcionalidades principales, segundo los problemas de accesibilidad que impidan la comprensión o el uso interactivo, y tercero los riesgos de integridad de datos. Tras implementar la corrección, es obligatorio repetir los pasos de reproducción y verificar que no se ha provocado una regresión en funciones adyacentes.

7. Plan de formación y revisión

La formación se plantea como una sesión breve seguida de práctica en el propio proyecto. El equipo puede distribuirla en tres bloques de aproximadamente 30–45 minutos, ajustando la duración a la disponibilidad y al nivel de experiencia.

7.1. Bloque 1: interfaz accesible

Revisar contraste, significado no dependiente del color, tipografía y uso de mayúsculas, mensajes integrados en vez de ventanas emergentes, teclado y foco. Actividad: auditar una pantalla real y proponer cambios verificables.

7.2. Bloque 2: recorrido de backend

Dibujar la petición, validación, permisos, almacenamiento y respuesta de una función. Actividad: analizar la creación de una tarea y enumerar casos válidos, inválidos, no autorizados y de fallo técnico.

7.3. Bloque 3: calidad compartida

Practicar criterios de aceptación, revisión entre compañeros, pruebas, registro de incidencias y criterio de terminado. Actividad: revisar un cambio pequeño con la lista de comprobación del manual.

7.4. Ritmo de revisión

Para cada funcionalidad nueva o modificada, la persona responsable de implementarla completa primero una auto-revisión. Después, otra persona del equipo revisa los criterios de aceptación y las pruebas. Julián apoya la revisión de accesibilidad y Yeray la de calidad; ambos pueden pedir cambios y registrar puntos pendientes, pero los criterios son responsabilidad de todo el equipo.

Cuando una decisión dependa del contexto del alumnado o del personal del centro, se solicitará validación al centro antes de darla por cerrada. Las pruebas automáticas o internas no sustituyen esa validación.

8. Responsabilidades y colaboración

Responsabilidad	**Funciones en este manual y en el trabajo diario**
Julián Carrión Tovar	
Responsable de accesibilidad	Mantener y ampliar las pautas de accesibilidad; ayudar a revisar colores, textos, interacción y alternativas; facilitar ejercicios de revisión y registrar problemas de accesibilidad.
Yeray Rodríguez Navas	
Responsable de calidad	Mantener los criterios de calidad; promover revisiones y pruebas; ayudar a clasificar incidencias y comprobar que las correcciones cumplen los criterios acordados.
Resto del equipo	Aplicar las pautas desde el diseño hasta la entrega, revisar cambios de compañeros, aportar pruebas y avisar cuando una decisión no esté clara o no pueda verificarse.
Equipo completo	Acordar prioridades y criterios; validar las decisiones que afecten a las personas usuarias; registrar pendientes y actualizar este manual cuando cambie el alcance.

8.1. Cómo colaborar ante un desacuerdo

Describir el problema observable, a quién puede afectar y qué criterio o necesidad está implicado. Proponer una alternativa y una forma de comprobarla. Si el equipo no puede resolverlo con la información disponible, registrar la decisión como pendiente y consultar a las personas responsables o al centro; no sustituir una necesidad no confirmada por una suposición.

9. Lista de comprobación y mejora continua

Antes de considerar terminada una funcionalidad, completar esta lista. Marcar “no aplica” solo cuando se anote una explicación breve.

9.1. Interfaz y accesibilidad

- El texto tiene contraste suficiente con su fondo y el contraste se ha comprobado con una herramienta.
- La información no depende solo del color, el sonido o un icono sin etiqueta.

- Los textos son legibles, directos y consistentes; las mayúsculas se limitan a rótulos breves.
- No hay ventanas emergentes automáticas o innecesarias; los avisos aparecen en contexto.
- Los errores explican cómo continuar y no borran datos introducidos sin avisar.
- Los controles funcionan con teclado y muestran un foco visible; se ha comprobado la ampliación del texto.

9.2. Backend y calidad

- El servidor valida los datos recibidos y aplica las reglas de negocio.
- Cada operación comprueba los permisos necesarios y devuelve solo los datos requeridos.
- Los errores se gestionan de forma controlada, sin exponer información interna ni marcar como guardado lo que falló.
- Se han probado el flujo normal y los casos de error relevantes.
- Otra persona ha revisado el cambio y los criterios de aceptación se pueden verificar.
- Las incidencias, decisiones y limitaciones pendientes están registradas y tienen seguimiento.

9.3. Registro de formación y decisiones

El equipo anotará la fecha de cada sesión, asistentes, ejercicios realizados, dudas que requieran seguimiento y cambios acordados para el manual. Para cada decisión relevante se registrará qué se decidió, por qué, quién la validó y si queda pendiente confirmación del centro. No se incluirán en actas ni ejemplos datos personales reales del alumnado.

9.4. Referencias para ampliar la formación

- W3C, *Web Content Accessibility Guidelines (WCAG) 2.2*: `https://www.w3.org/TR/WCAG22/`.
- W3C WAI, comprensión del criterio de contraste mínimo: `https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html`.
- W3C WAI-ARIA Authoring Practices Guide, patrón de diálogo modal: `https://www.w3.org/WAI/ARIA/apg/patterns/dialog-modal/`.

Estas fuentes ayudan a concretar las comprobaciones técnicas; el equipo debe adaptarlas a las funciones reales del proyecto y contrastar la experiencia con sus usuarios previstos.